Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г. В. Плеханова»
МОСКОВСКИЙ ПРИБОРОСТРОИТЕЛЬНЫЙ ТЕХНИКУМ

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: Программист

ПРАКТИЧЕСКИЕ РАБОТЫ ПО ДИСЦИПЛИНЕ

«РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ»

Листов: 7

Выполнил студент

Группы П-2-24

Коротаев Д.И.

Проверил преподаватель

Осипян Л.Г

Москва 2026

Практическая работа №10

Слоистая архитектура и AutoMapper в ASP.NET Core Web API

Цель работы: перевести существующий Web API интернет-магазина на слоистую архитектуру, настроить AutoMapper и добавить резервирование товара с возможностью отмены.

Ход работы:

1. Разделение проекта на слои.

В проекте Apitest сохранено подключение к базе данных SQL Server shopBD через ShopBdContext. Для всех девяти сущностей созданы интерфейсы и реализации репозиториев и сервисов. Репозитории обращаются к базе данных, сервисы выполняют бизнес-логику, а контроллеры вызывают сервисы и возвращают HTTP-ответы. В Program.cs репозитории и сервисы зарегистрированы как Scoped. В пределах одного запроса они используют общий контекст базы данных. Это позволяет сохранить изменения нескольких связанных сущностей вместе.

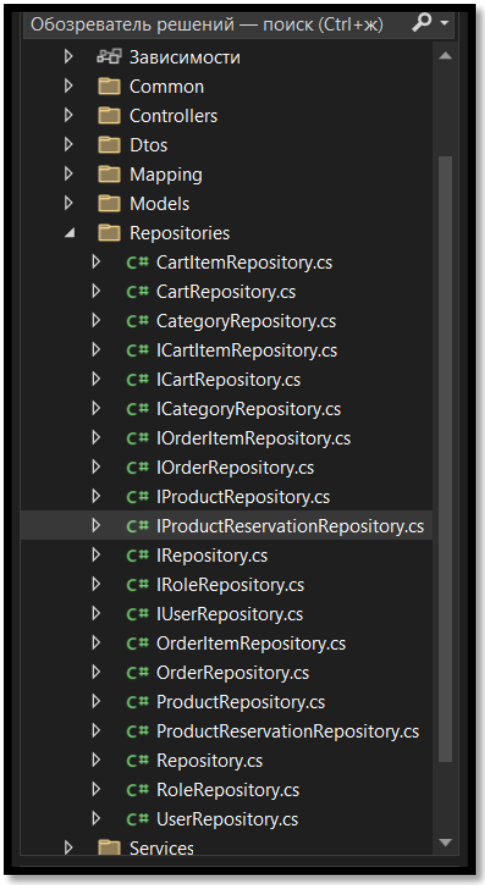


Рисунок 1 - Структура проекта со слоями Repository, Service и Controller.

2. Подключение AutoMapper и настройка DTO.

Установлен AutoMapper версии 15.1.3. В MappingProfile настроены преобразования сущностей в DTO и обратно. DTO чтения содержат вложенные связанные данные, а DTO записи принимают Id связанных сущностей. Для PATCH используются nullable-поля и условия PreCondition, поэтому пропущенные значения не изменяются.

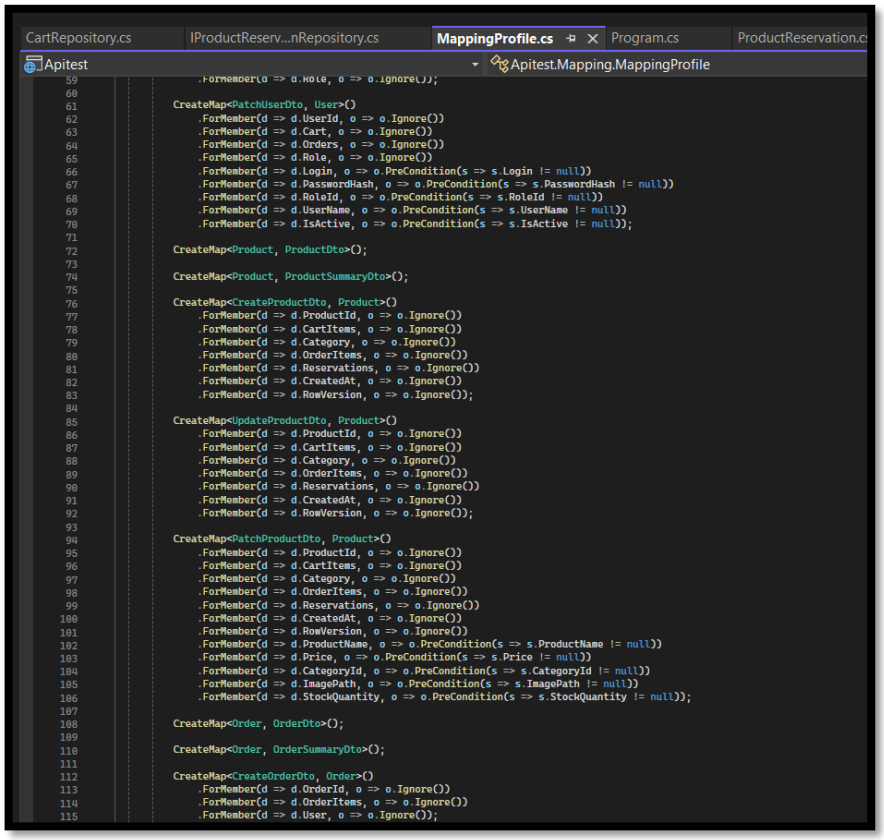


Рисунок 2 - Настройка преобразований в MappingProfile.

3. Реализация CRUD и пагинации.

Для всех сущностей реализованы получение списка и записи по Id, создание, полное и частичное обновление, удаление. Список сортируется по Id, после чего применяются Skip и Take. Параметры page и pageSize задают номер и размер страницы. В заголовке X-Total-Count передается общее количество записей. GET возвращает 200 OK, POST - 201 Created со ссылкой на созданную запись, PUT, PATCH и DELETE - 204 No Content. Если запись не найдена, возвращается 404 Not Found. В DTO товара выводятся данные связанной категории.

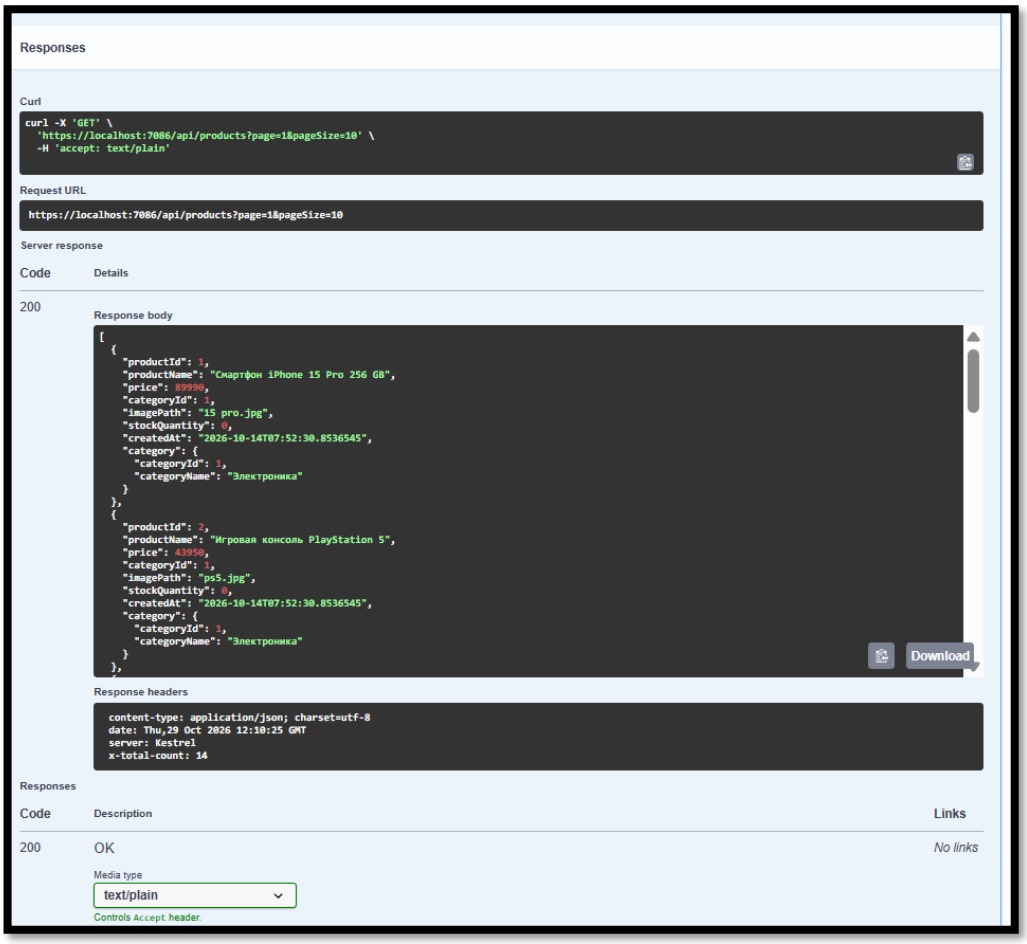


Рисунок 3 - Получение списка товаров с категорией и заголовком X-Total-Count.

Полное обновление PUT принимает все поля DTO записи. PATCH изменяет только переданные поля. Проверки входных данных задаются через Data Annotations и сервисы. Ручные методы преобразования ToDTO больше не используются.

4. Резервирование товара и проверка остатка.

В сущность Product добавлено поле StockQuantity. Создана сущность ProductReservation, которая хранит пользователя, товар, количество, время создания и отмены резерва. Между товаром и резервами настроена связь один-ко-многим. Метод POST /api/productreservations проверяет наличие товара и активного пользователя, а также доступное количество. При создании резерва уменьшается StockQuantity и добавляется запись ProductReservation. Оба изменения сохраняются одним вызовом SaveChangesAsync. Например, при резервировании 3 единиц из 10 доступный остаток становится равен 7.

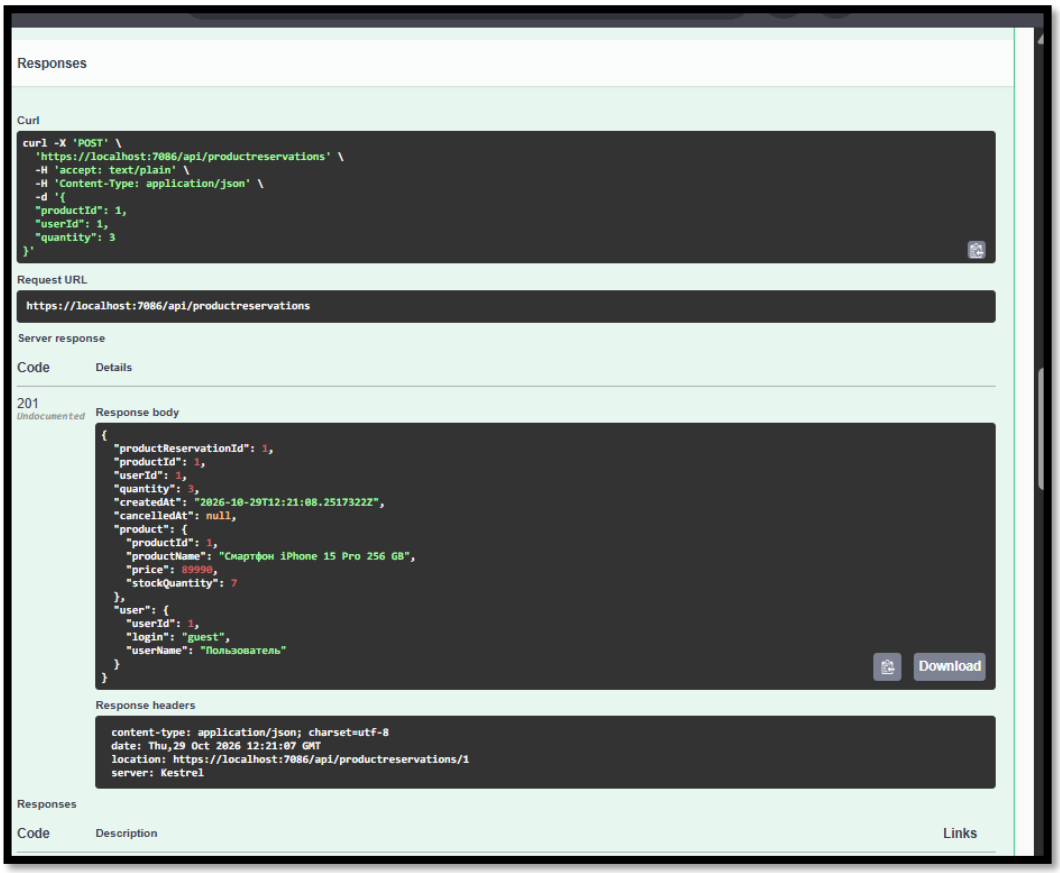


Рисунок 4 - Создание резерва и уменьшение остатка товара.

Если доступно 7 единиц товара, запрос на резервирование 8 единиц отклоняется с кодом 400 Bad Request и сообщением о недостаточном остатке. В этом случае резерв не создается и количество товара не изменяется.



Рисунок 5 - Отказ в резервировании при недостаточном остатке.

5. Отмена резервирования и обработка ошибок.

Метод POST /api/productreservations/{id}/cancel находит активный резерв, возвращает его количество в доступный остаток товара и заполняет CancelledAt. Изменение товара и закрытие резерва сохраняются одним вызовом SaveChangesAsync, без отдельного UnitOfWork и явных транзакций. После отмены резерва на 3 единицы остаток восстанавливается с 7 до 10.

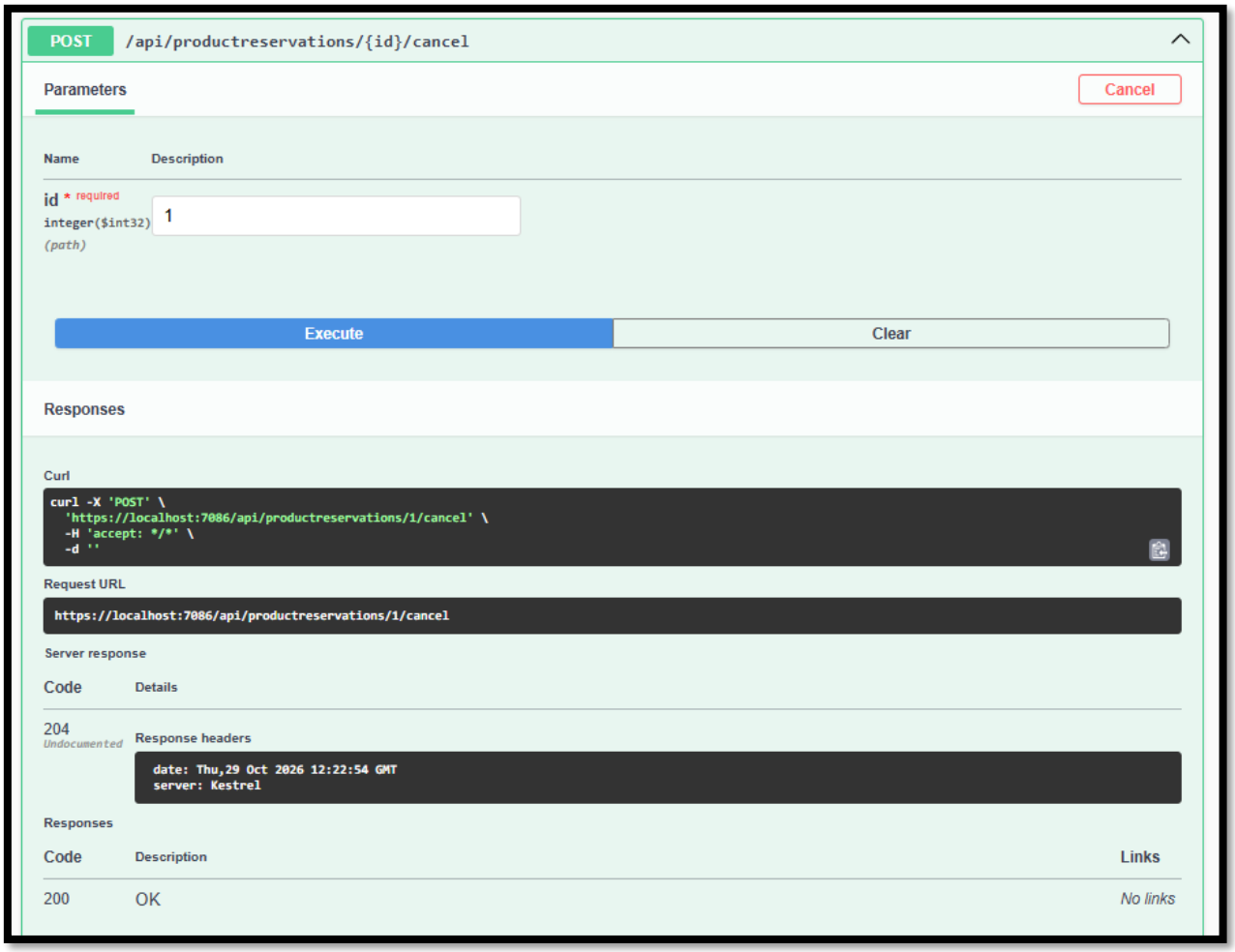


Рисунок 6 - Закрытый резерв и восстановленный остаток товара.

При повторной отмене этого же резерва возвращается 404 Not Found с сообщением об отсутствии активного резерва. Повторный возврат товара не выполняется. Ошибки сервисов переводятся в HTTP-ответы через ServiceExceptionFilter.

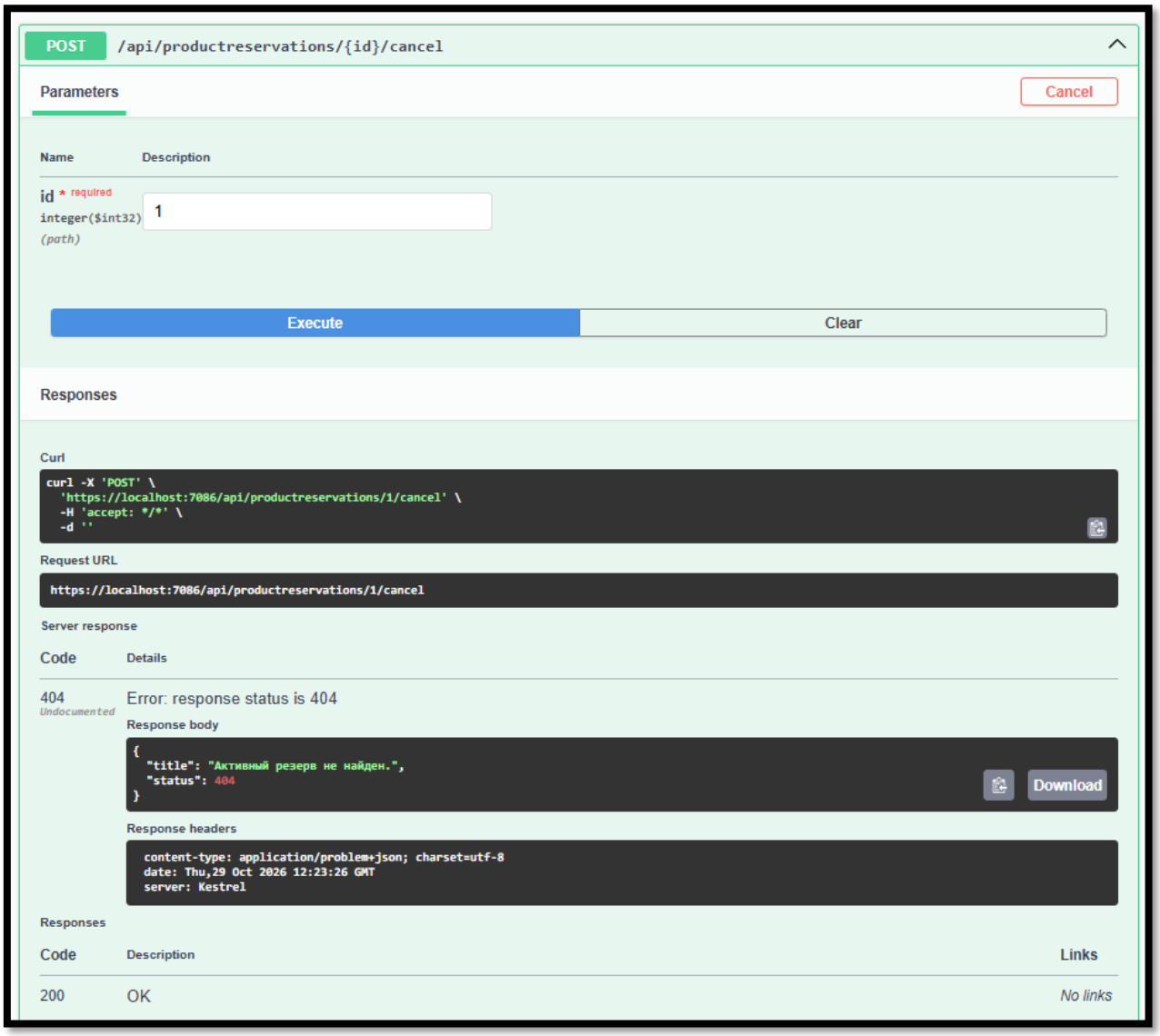


Рисунок 7 - Ответ 404 при повторной отмене резерва.

Вывод: в ходе практической работы проект Apitest переведен на слоистую архитектуру. Настроен AutoMapper для работы с DTO, сохранены CRUD-операции для всех сущностей. Добавлены резервирование товара и его отмена с проверкой остатка и сохранением связанных изменений одним вызовом SaveChangesAsync.